

MCP Server Layer — apm_connectors_mcp/

A from-basics walkthrough of the separate package that exposes the /tools/* API as MCP tools for an LLM-based agent — the natural-language front door, and why it needed no new enforcement code.

1. The problem this layer solves — a structural fact worth noticing first

Every layer covered so far (FastAPI, Action Graph, Connector Layer, State Store, Terraform) lives inside one Python package, `apm_connectors`. This layer does not. It's `src/apm_connectors_mcp/` — a **separate top-level package**, with its own `mcp` extra in `pyproject.toml` (`mcp = ["mcp>=2.0", "httpx>=0.27"]`) and its own console-script entry point (`apm-connectors-mcp = "apm_connectors_mcp.server:main"`). That's not incidental organization — it's the clearest physical evidence yet of the principle stated in this repo's `CLAUDE.md`: “connectors stay reasoning-free... whatever reasoning layer consumes them is deployed separately.” This package can be installed, run, and executed on a completely different machine from the /tools/* API server it talks to, with a completely different, much smaller dependency set (no `googleapiclient`, no `requests`, no `langgraph` — just `httpx` and `mcp`).

The problem it solves: an LLM-based agent host (Claude Desktop, Claude Code, or any MCP-capable client) doesn't speak raw HTTP + hand-parsed JSON. It speaks **MCP** — a protocol specifically designed for an LLM to discover a set of callable “tools,” each with a name, a typed schema, and a natural-language description, and decide on its own which to call based on a free-text request. This layer's whole job is exposing the exact same /tools/* contract already covered, through that different vocabulary, to that different kind of caller.

2. MCP basics, since this builds on it

MCP (Model Context Protocol) is a standard for connecting an LLM-based agent host to external capabilities. Three ideas matter here:

Concept	What it means
Tool	A callable action a server exposes — name, typed parameters, a docstring, a declared return shape. Unlike a plain Python function, an MCP tool's schema and description are read by the LLM at decision time to decide whether and how to call it.
Server / Host	The server (this package) exposes tools; the host (Claude Desktop, Claude Code, etc.) is the LLM-facing application that connects to servers and lets the model call their tools.
Transport	How host and server actually talk — <code>stdio</code> (local subprocess, <code>stdin/stdout</code> ; the default for a locally-configured agent host) or <code>sse/streamable-http</code> (a network service instead, for a remote reasoning layer).

A minimal MCP tool — the pattern this file repeats eighteen times

```
@mcp.tool()
async def my_tool(x: int) -> dict:
    """A description the LLM reads to decide when/how to call this."""
    return {"result": x * 2}
```

The framework introspects the function signature for the parameter schema and uses the docstring as the tool's description — both become part of what the connected LLM actually sees when deciding what to call. Worth sitting with, because it's the single most distinctive thing about this layer versus every other one covered so far (§5).

3. Pattern: a pure HTTP adapter, never touching the layers it fronts

"Thin async HTTP client for apm_connectors' /tools/ API — the only thing apm_connectors_mcp talks to. Deliberately dumb: it never imports apm_connectors.tools/graph directly, so this package can run anywhere with just httpx + mcp installed, against a /tools/* server deployed wherever."*

ConnectorClient (client.py) is genuinely minimal — one constructor, one post method, ~35 lines of real logic. It has no knowledge of GmailTool, BaseTool, graph.py, or StateStore. Every one of the other five layers this document set has covered is, from this package's point of view, just "whatever's running behind APM_CONNECTORS_BASE_URL." This is the same "depend on the API contract, not the implementation" discipline the State Store reference called a *migration seam* — here it's drawn at a much bigger scale: not just swappable storage, but a swappable, independently-deployable *process*.

```
class ConnectorClient:
    def __init__(self, base_url: str | None = None, transport: httpx.AsyncBaseTransport | None =
        None) -> None:
        self._client = httpx.AsyncClient(
            base_url=base_url or os.environ.get("APM_CONNECTORS_BASE_URL", DEFAULT_BASE_URL),
            timeout=30.0,
            transport=transport,
        )
```

Two configuration surfaces worth naming: `base_url` defaults to `APM_CONNECTORS_BASE_URL`, falling back to `127.0.0.1:8000` (the same default `docs/running-locally.md` uses) — so a locally-run MCP server talks to a locally-run API server with zero configuration, but pointing it at a remote deployment (say, the ECS Fargate `service_url` from the Terraform reference) is a single environment variable, no code change. `transport` is the testing hook — covered fully in §9.

4. Pattern: a factory function, again, for the same reason as every other layer

```
def build_server(client: ConnectorClient, name: str = "apm-connectors") -> MCPServer:
    """Registers every /tools/* route as an MCP tool against `client`.
    A factory rather than a module-level singleton so tests (or a
    caller that wants a non-default ConnectorClient) can inject one --
    same shape as apm_connectors.graph.build_action_graph(tools, ...)."""
    mcp = MCPServer(name)
    ...
    return mcp
```

By now this should look familiar: `dependencies.py`'s `get_tools()/get_action_graph()` were factories taking dependencies as parameters; `graph.py`'s `build_action_graph(tools, state_store, checkpointer)` was the same shape; here it's `build_server(client)`. Same recurring need across every layer in this codebase — swap what's plugged in without touching the logic that uses it — expressed once more, this time for an `MCPServer` instance instead of a FastAPI app or a compiled graph.

5. Pattern: the docstring as a machine-consumed interface, not just documentation

This is the pattern genuinely unique to this layer. Compare two docstrings side by side:

`gmail_tool.py` (Connector Layer) — documentation for a human reading the source

```
def search_emails(self, process_id: str, query: str, max_results: int = 10) ->
    list[EmailSummary]:
    """Search the mailbox using Gmail's query syntax..."""
```

server.py (this layer) — read by the LLM agent deciding whether/how to call this

```
@mcp.tool()
async def gmail_search(query: str = "", max_results: int = 10, process_id: str | None = None) ->
    list[dict]:
    """Search Gmail. `query` is free text -- Gmail's search syntax
    works if used ("from:x@y.com", "newer_than:7d", "is:unread",
    "subject:invoice"), but plain keywords work too, same as
    typing into the Gmail search box. Pass "" (the default) for no
    filter at all -- the most recent `max_results` messages.
    Read-only: executes immediately, no approval needed.
    """
```

“Tool descriptions below are deliberately detailed (Gmail's query syntax, RFC3339 datetime format, action_id vs. process_id) — this is the vocabulary an LLM-based agent uses to translate a free-text request (‘what's the status of order 401’) into an actual call (gmail_search(query='order 401')) on its own, which is the whole point of exposing these as MCP tools instead of a raw HTTP contract a hand-written intent parser would have to keep in sync by hand.”

This reframes what a docstring is, at this layer specifically. Everywhere else in this codebase, a docstring is for the next engineer reading the file. Here, it's runtime-consumed input to a model's decision about whether to call this tool and with what arguments — closer to a prompt than to a comment. That's also why every write tool's docstring repeats the same explicit warning almost verbatim (“This does NOT send anything — it pauses for human approval”) rather than relying on a shared reference: an LLM reads *this* tool's description in isolation when deciding how to act, so the safety-relevant fact has to be restated locally, every time.

6. Pattern: the approval gate, re-verified from the outside

“Every write tool... mirrors the REST API exactly: it does not execute anything — it returns a paused action_id, and the agent must call decide_action(action_id, approved=True) to actually run it. That gate is enforced server-side in the /tools/ API regardless of what this MCP layer does, so it can't be bypassed by a misbehaving or adversarial agent — this module is a description/transport convenience on top of it, not a second copy of the guardrail.”*

Look at what gmail_send actually does here:

```
@mcp.tool()
async def gmail_send(to: str, subject: str, body: str, process_id: str | None = None) ->
    dict[str, Any]:
    """Propose sending a Gmail email. This does NOT send anything..."""
    return await _call("/tools/gmail/send", {"to": to, "subject": subject, "body": body,
    "process_id": process_id})
```

It's an HTTP POST to /tools/gmail/send — the exact same route covered in the FastAPI reference, hitting the exact same _propose() → start_action() → propose_node → approval_node[interrupt()] pipeline covered in the Action Graph reference. **This layer adds no new enforcement, and needs none.** An “adversarial agent” — one that hallucinates, or is deliberately prompted to try to send an email without approval — has no more power here than a malicious script hitting the REST API directly, because both go through the identical server-side gate. Nothing about exposing these as MCP tools required re-deriving or re-implementing that guarantee.

7. Pattern: error translation — a fourth link in a chain you've seen at every layer

Trace one failure all the way through, since you've now seen each individual link in earlier references:

```
# apm_connectors/api/_responses.py (FastAPI layer)
def upstream_error(exc: Exception) -> HTTPException:
    return HTTPException(status_code=502, detail=f"Upstream tool error: {exc}")

# apm_connectors_mcp/client.py (this layer)
class ConnectorAPIError(Exception): ...

async def post(self, path: str, body: dict[str, Any]) -> Any:
    response = await self._client.post(path, json=body)
    if response.status_code >= 400:
        raise ConnectorAPIError(response.status_code, _extract_detail(response))
    return response.json()

# apm_connectors_mcp/server.py (this layer)
async def _call(path: str, body: dict[str, Any]) -> Any:
    try:
        return await client.post(path, body)
    except ConnectorAPIError as exc:
        # ToolError is what MCP converts into a clean is_error result
        # for the agent to see, rather than a raw/uncaught exception.
        raise ToolError(str(exc)) from exc
```

Four translation boundaries, four different exception types, each owned by the layer it protects: a raw Python exception inside a connector → HTTPException at the FastAPI boundary → ConnectorAPIError at the HTTP-client boundary → ToolError at the MCP boundary, which is what the *agent itself* ultimately sees as a clean `is_error` result — never a stack trace, never an unhandled crash, at any point in that chain. This is the pattern's fourth and final appearance, closing the loop from the deepest connector call all the way out to the LLM agent's own view of what happened.

8. Pattern: `decide_action` — the MCP-layer twin of the one shared decision route

```
@mcp.tool()
async def decide_action(action_id: str, approved: bool) -> dict[str, Any]:
    """Approve or reject a pending write proposed by gmail_send,
    calendar_create_event, excel_write, salesforce_create/update, or
    jira_create/update (its action_id from that call's response).
    Nothing in the real system happens until this is called with
    approved=true; approved=false discards it -- nothing is
    sent/created/written either way.
    """
    return await _call(f"/tools/actions/{action_id}/decision", {"approved": approved})
```

Exactly one `decide_action` tool, same as exactly one shared POST `/tools/actions/{action_id}/decision` route in `tools_routes.py` — this layer really is a mechanical, one-tool-per-route mirror of the REST contract, and that claim is directly tested, not just asserted:

```

async def test_lists_one_tool_per_tools_route(tmp_path: Path) -> None:
    mcp, *_ = _mcp(tmp_path)
    tools = await mcp.list_tools()
    names = {t.name for t in tools}
    assert names == {"gmail_search", "gmail_read", "gmail_send", ..., "decide_action"}

```

Eighteen tools, one call to `mcp.list_tools()`, compared against the exact expected set — the clearest possible proof that this layer adds no hidden capabilities and drops none.

9. Pattern: testing through the real protocol layer, over an in-process transport

This composes substitution tricks from *three different layers at once* — the most elaborate testing setup in the whole codebase:

```

def _mcp(tmp_path: Path, ...):
    # 1. Connector layer: fake clients, no live credentials
    tools = {"gmail": GmailTool(store, FakeGmailClient(...)), ...}
    action_graph = build_action_graph(tools, store, checkpoint=MemorySaver())

    # 2. FastAPI layer: dependency_overrides swap in the fake-backed tools/graph
    app = create_app()
    app.dependency_overrides[get_tools] = lambda: tools
    app.dependency_overrides[get_action_graph] = lambda: action_graph

    # 3. This layer: ASGITransport swaps "a real HTTP socket" for
    #    "call straight into the ASGI app object in-process"
    client = ConnectorClient(base_url="http://testserver",
                           transport=httpx.ASGITransport(app=app))
    mcp = build_server(client)
    return mcp, ...

```

`httpx.ASGITransport` is the piece unique to this layer: normally, `httpx.AsyncClient` sends real bytes over a real socket to a real running server. Pass it an `ASGITransport(app=app)` instead, and every `client.post(...)` call — from `ConnectorClient`, unmodified — is routed directly into the FastAPI app object's ASGI interface in-process, no socket, no separately-running uvicorn, no network at all. Combined with `dependency_overrides` and `MemorySaver()`, the test exercises the *actual* MCP protocol layer, the *actual* FastAPI route handlers and Pydantic validation, and the *actual* `graph.py` propose/approve/execute pipeline — genuinely everything except a live socket and live third-party credentials. Three layers, three different substitution mechanisms (`Depends()` overrides, closures/factories, transport injection), the same underlying idea each time.

10. Walkthrough: one MCP tool call, traced across all five layers

`gmail_send("customer@realcorp.io", "Update", "Your order shipped")`, called by an LLM agent that decided this was the right tool for a user's request:

1	MCP	The host serializes the call per the MCP protocol; <code>build_server</code> 's registered <code>gmail_send</code> closure runs, calls <code>_call("/tools/gmail/send", {...})</code> .
2	MCP → HTTP	<code>ConnectorClient.post</code> sends a real (or in-process, in tests) POST <code>/tools/gmail/send</code> .
3	FastAPI	<code>gmail_send</code> route: <code>_tool(tools, "gmail")</code> fail-fast check, then <code>_propose() → start_action()</code> .

4	Action Graph	propose_node records a pending action (State Store: add_pending_action, logs "action_proposed"); approval_node hits interrupt(), the graph pauses.
5	FastAPI → MCP	RunOutcomeResponse with pending_action set flows back as JSON; gmail_send hands it back as structured_content — action_id + pending_action visible to the agent, nothing sent.
6	MCP (later)	Agent (or the human behind it) calls decide_action(action_id, approved=True).
7	FastAPI	decide_action route → resume_process() → Command(resume={"approved": True}) into the same cached graph thread.
8	Action Graph	approval_node replays, interrupt() returns immediately, State Store: resolve_pending_action logs "action_approved"; execute_node runs, decision=True → real branch: GmailTool.send_email(..., dry_run=False) fires — Connector Layer's placeholder-domain guard already passed, require_dry_run_guard logs "action_executed".
9	All the way back	final_result with executed: true flows back through FastAPI → ConnectorClient → decide_action's return value → the agent sees the email was actually sent.

Nine steps, five layers, one gate (step 4/8) that nothing in this chain has the power to skip.

11. Configuration and transport surface

```
pip install -e ".[mcp]"
APM_CONNECTORS_BASE_URL=http://127.0.0.1:8000 apm-connectors-mcp
```

```
def main() -> None:
    client = ConnectorClient()
    mcp = build_server(client)
    transport = os.environ.get("APM_CONNECTORS_MCP_TRANSPORT", "stdio")
    mcp.run(transport=transport)
```

Two environment variables control everything: APM_CONNECTORS_BASE_URL (which /tools/* server to talk to — defaults to local) and APM_CONNECTORS_MCP_TRANSPORT (stdio by default, matching a locally-configured agent host like Claude Desktop/Code; sse or streamable-http for a remote reasoning layer connecting over the network instead). No other configuration surface exists.

12. Full tool inventory

MCP tool	Kind	REST route
gmail_search, gmail_read	read	POST /tools/gmail/{search,read}
gmail_send	write	POST /tools/gmail/send
calendar_search, calendar_read	read	POST /tools/calendar/{search,read}
calendar_create_event	write	POST /tools/calendar/create-event
excel_worksheets, excel_read	read	POST /tools/excel/{worksheets,read}
excel_write	write	POST /tools/excel/write
salesforce_query, salesforce_read	read	POST /tools/salesforce/{query,read}
salesforce_create, salesforce_update	write	POST /tools/salesforce/{create,update}

jira_search, jira_read	read	POST /tools/jira/{search,read}
jira_create, jira_update	write	POST /tools/jira/{create,update}
decide_action	shared decision	POST /tools/actions/{action_id}/decision

18 tools, exactly mirroring the FastAPI reference's route table — this is the same contract, in a different vocabulary, for a different kind of caller.

13. Vocabulary cheat-sheet

Term	Meaning here
MCP (Model Context Protocol)	A standard for an LLM-based agent host to discover and call tools exposed by a separate server process.
Tool (MCP sense)	A callable action with a typed schema and a description — both read by the LLM at call-decision time, not just by a human reading source.
Host	The LLM-facing application (Claude Desktop, Claude Code, etc.) that connects to MCP servers and lets the model call their tools.
Transport	How host and server communicate — stdio (local subprocess) or sse/streamable-http (network service).
httpx.ASGITransport	Routes an httpx client's calls directly into an ASGI app object in-process, no socket — the testing hook that lets this layer's tests exercise real FastAPI route handlers with no live server.
structured_content	The typed result payload an MCP tool call returns to the calling agent.
ToolError / is_error	MCP's mechanism for surfacing a failure to the agent as a clean, structured error result instead of a raw exception.

That's the whole layer — two small files, ~380 lines total — and it adds exactly one thing to everything covered before it: a natural-language-shaped front door for an LLM agent, sitting entirely outside the trust boundary, with zero new privilege and zero new enforcement code, because the one guarantee that matters — a write cannot execute without a separate, later, human-approved call — was already sitting one layer down, and works identically no matter what's calling it.